ARTI308 - Machine Learning

# Logistic Regression Project - Solutions

In this project we will be working with a fake advertising data set, indicating whether or not a particular internet user clicked on an Advertisement on a company website. We will try to create a model that will predict whether or not they will click on an ad based off the features of that user.

This data set contains the following features:

- 'Daily Time Spent on Site': consumer time on site in minutes
- 'Age': cutomer age in years
- 'Area Income': Avg. Income of geographical area of consumer
- 'Daily Internet Usage': Avg. minutes a day consumer is on the internet
- 'Ad Topic Line': Headline of the advertisement
- 'City': City of consumer
- 'Male': Whether or not consumer was male
- 'Country': Country of consumer
- 'Timestamp': Time at which consumer clicked on Ad or closed window
- 'Clicked on Ad': 0 or 1 indicated clicking on Ad

## Import Libraries

**Import a few libraries you think you'll need (Or just import them as you go along!)**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
```

## Get the Data

**Read in the advertising.csv file and set it to a data frame called ad_data.**

Gemini

```
import os
import urllib.request

# Download the file if it does not exist in the workspace
# Download the file from a working URL if it doesn't exist
if not os.path.exists('advertising.csv'):
    print("Downloading advertising.csv...")
    url = 'https://raw.githubusercontent.com/singhsidhant/Logistic-Regression-Project/master/advertising.csv'
    urllib.request.urlretrieve(url, 'advertising.csv')
    url = 'https://raw.githubusercontent.com/yanshengjia/ml-by-stanford/master/Logistic%20Regression/advertising.
    try:
        urllib.request.urlretrieve(url, 'advertising.csv')
        print("Download complete!")
    except Exception as e:
        print(f"Failed to download: {e}. Please upload the file manually.")

ad_data = pd.read_csv('advertising.csv')
if os.path.exists('advertising.csv'):
    ad_data = pd.read_csv('advertising.csv')
    display(ad_data.head())
```

| | Daily Time Spent on Site | Age | Area Income | Daily Internet Usage | Ad Topic Line | City | Male | Country | Timestamp | Clicked on Ad |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 68.95 | 35 | 61833.90 | 256.09 | Cloned 5thgeneration orchestration | Wrightburgh | 0 | Tunisia | 2016-03-27 00:53:11 | 0 |
| 1 | 80.23 | 31 | 68441.85 | 193.77 | Monitored national standardization | West Jodi | 1 | Nauru | 2016-04-04 01:39:02 | 0 |
| 2 | 69.47 | 26 | 59785.94 | 236.50 | Organic bottom-line service-desk | Davidton | 0 | San Marino | 2016-03-13 20:35:42 | 0 |

**Check the head of ad_data**

```
ad_data.head()
```

| | Daily Time Spent on Site | Age | Area Income | Daily Internet Usage | Ad Topic Line | City | Male | Country | Timestamp | Clicked on Ad |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 68.95 | 35 | 61833.90 | 256.09 | Cloned 5thgeneration orchestration | Wrightburgh | 0 | Tunisia | 2016-03-27 00:53:11 | 0 |
| 1 | 80.23 | 31 | 68441.85 | 193.77 | Monitored national standardization | West Jodi | 1 | Nauru | 2016-04-04 01:39:02 | 0 |
| 2 | 69.47 | 26 | 59785.94 | 236.50 | Organic bottom-line service-desk | Davidton | 0 | San Marino | 2016-03-13 20:35:42 | 0 |

Next steps: Generate code with ad_data | New interactive sheet

** Use info and describe() on ad_data**

```
ad_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Daily Time Spent on Site  1000 non-null   float64
 1   Age                       1000 non-null   int64  
 2   Area Income               1000 non-null   float64
 3   Daily Internet Usage      1000 non-null   float64
 4   Ad Topic Line             1000 non-null   object 
 5   City                      1000 non-null   object 
 6   Male                      1000 non-null   int64  
 7   Country                   1000 non-null   object 
 8   Timestamp                 1000 non-null   object 
 9   Clicked on Ad             1000 non-null   int64  
dtypes: float64(3), int64(3), object(4)
memory usage: 78.3+ KB
```

```
ad_data.describe()
```

| | Daily Time Spent on Site | Age | Area Income | Daily Internet Usage | Male | Clicked on Ad |
|---|---|---|---|---|---|---|
| count | 1000.000000 | 1000.000000 | 1000.000000 | 1000.000000 | 1000.000000 | 1000.00000 |
| mean | 65.000200 | 36.009000 | 55000.000080 | 180.000100 | 0.481000 | 0.50000 |
| std | 15.853615 | 8.785562 | 13414.634022 | 43.902339 | 0.499889 | 0.50025 |
| min | 32.600000 | 19.000000 | 13996.500000 | 104.780000 | 0.000000 | 0.00000 |
| 25% | 51.360000 | 29.000000 | 47031.802500 | 138.830000 | 0.000000 | 0.00000 |
| 50% | 68.215000 | 35.000000 | 57012.300000 | 183.130000 | 0.000000 | 0.50000 |
| 75% | 78.547500 | 42.000000 | 65470.635000 | 218.792500 | 1.000000 | 1.00000 |
| max | 91.430000 | 61.000000 | 79484.800000 | 269.960000 | 1.000000 | 1.00000 |

## Exploratory Data Analysis

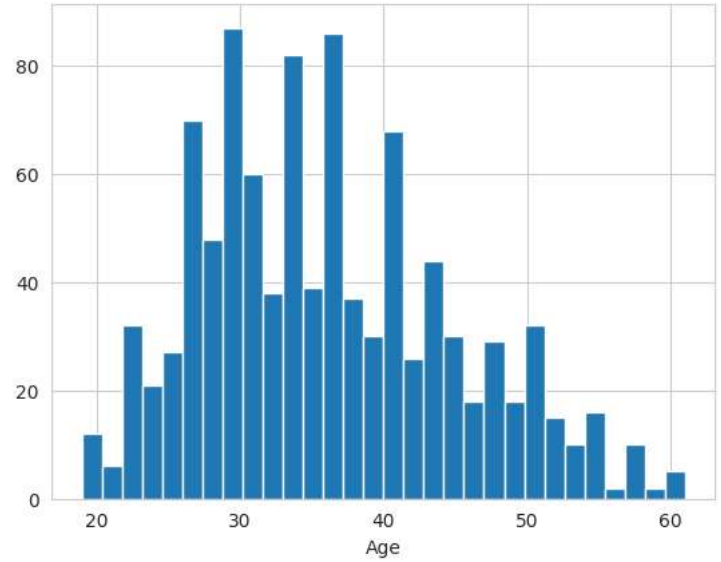
Let's use seaborn to explore the data!

Try recreating the plots shown below!

** Create a histogram of the Age**

```
sns.set_style('whitegrid')
ad_data['Age'].hist(bins=30)
plt.xlabel('Age')
```

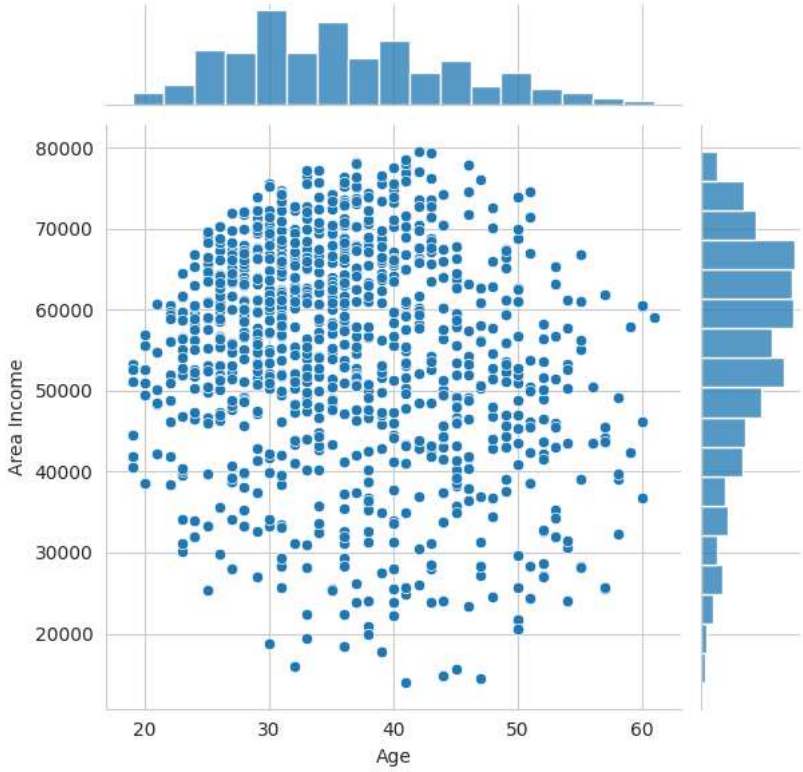
```
Text(0.5, 0, 'Age')
```




**Create a jointplot showing Area Income versus Age.**

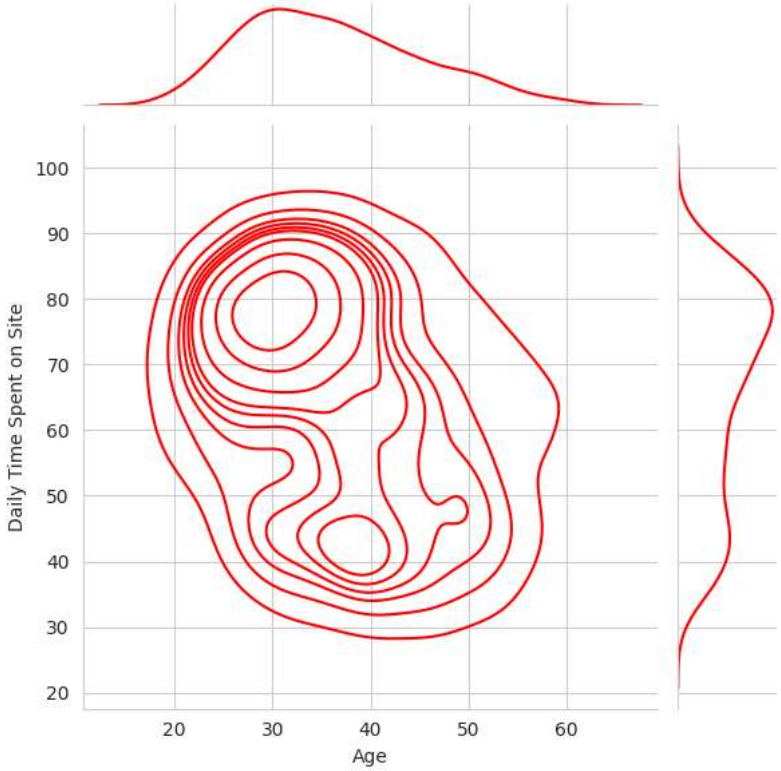
```
sns.jointplot(x='Age', y='Area Income', data=ad_data)
```

```
<seaborn.axisgrid.JointGrid at 0x7bf85b600590>
```




**Create a jointplot showing the kde distributions of Daily Time spent on site vs. Age.**

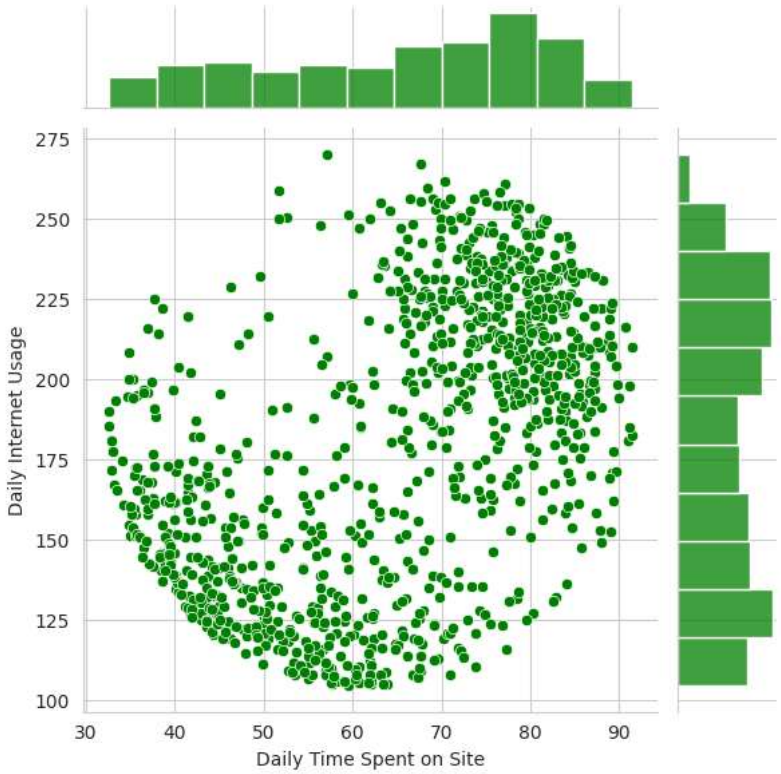
```
sns.jointplot(x='Age', y='Daily Time Spent on Site', data=ad_data, color='red', kind='kde');
```




** Create a jointplot of 'Daily Time Spent on Site' vs. 'Daily Internet Usage'**

```
sns.jointplot(x='Daily Time Spent on Site', y='Daily Internet Usage', data=ad_data, color='green')
```

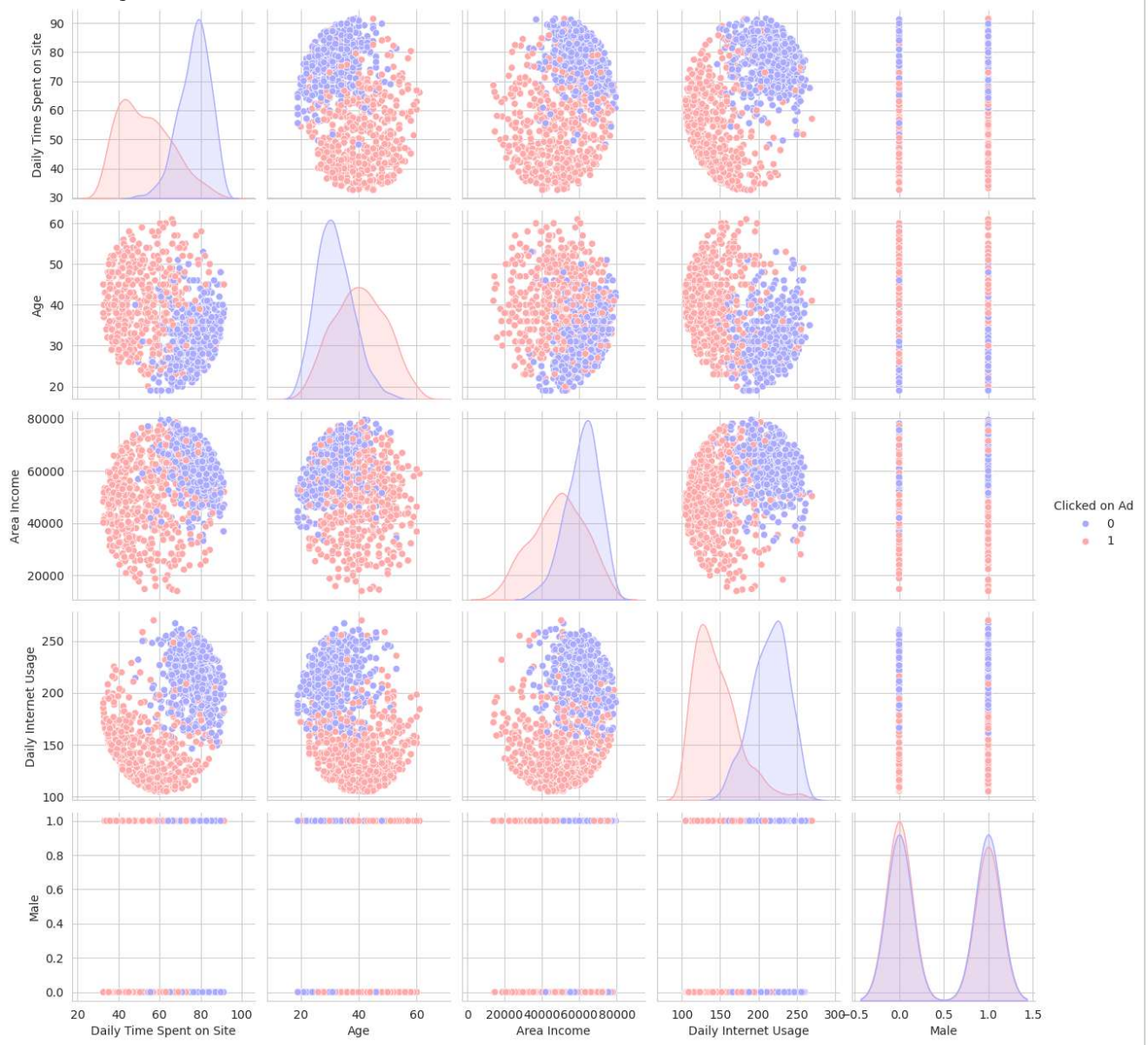
```
<seaborn.axisgrid.JointGrid at 0x7bf858194190>
```




** Finally, create a pairplot with the hue defined by the 'Clicked on Ad' column feature.**

```
sns.pairplot(ad_data, hue='Clicked on Ad', palette='bwr')
```

```
<seaborn.axisgrid.PairGrid at 0x7bf85b601400>
```




## Logistic Regression

Now it's time to do a train test split, and train our model!

You'll have the freedom here to choose columns that you want to train on!

** Split the data into training set and testing set using train_test_split**

```
X = ad_data[['Daily Time Spent on Site', 'Age', 'Area Income', 'Daily Internet Usage', 'Male']]
y = ad_data['Clicked on Ad']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
```

** Train and fit a logistic regression model on the training set.**

```
logmodel = LogisticRegression(max_iter=1000)
logmodel.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=1000)
```

```
Start coding or generate with AI.
```

## Predictions and Evaluations

** Now predict values for the testing data.**

```
predictions = logmodel.predict(X_test)
```

** Create a classification report for the model.**

```
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.96      0.98      0.97       162
           1       0.98      0.96      0.97       168

    accuracy                           0.97       330
   macro avg       0.97      0.97      0.97       330
weighted avg       0.97      0.97      0.97       330
```

```
Start coding or generate with AI.
```

## Great Job!